

MC simulations of structural transitions in a 2D system

In this project, we will study how a variety of structural phases can appear in a 2D system, even though the particles only interact via an extremely simple potential. This system was originally studied by Manelscio and Pellicane, in a work that was published in *Nature Materials* [Malescio, Pellicane, *Nature Mater.* **2**, 97 (2003)].

We will use the Metropolis Monte Carlo Method, and apply 2D periodic boundary conditions. The interaction energy between two particles, separated a distance r , is described by the following potential, $\Phi(r)$:

$$\phi(r) = \begin{cases} \infty, & r < \sigma_0 \\ \epsilon, & \sigma_0 < r < \sigma_1 \\ 0, & r > \sigma_1 \end{cases} \quad (1)$$

In this project, we have set $\sigma_1 = 2.5\sigma_0$.

This pair potential, and the particles, are illustrated in Figure 1.

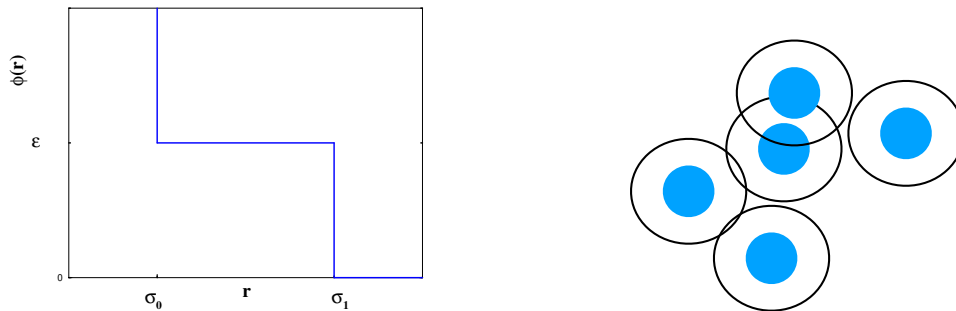


Figure 1: The pair potential, $\Phi(r)$, and a cartoon illustration of the particles, which carry a soft and a hard core.

We will utilise the canonical NVT -ensemble, in which the “volume” (area) V , the temperature T and the number of particles N are fixed, i.e. input parameters.

Equilibration and production

Even though other options exist, it is often beneficial to split up the total simulation effort into one (or several) “equilibrium run” and a subsequent (usually longer) production run(s). The goal of the equilibrium run is, as the name suggests, to establish equilibrium and produce a configuration that represents a typical equilibrium state. The final (x, y) coordinates from this simulation, is thus written to disk, to be use as the starting configuration for the next (production) simulation.

At the start of the equilibrium simulation, we obviously need to provide a starting configuration. There are two common choices: a completely random configuration (“hot start”) or an ordered arrangement (“cold start”). The approach to equilibrium can be conveniently traced by monitoring the energy as a function of the number of attempted configurations (MC steps), as described in the lecture notes. Note that a random (hot) start usually involves overlap between the central cores of some particles and thus in principle infinite energy. This is not really a problem, provided that we make sure that such overlaps vanish during the approach to equilibrium.

One way to ensure this is to initially run a pre-equilibration at an elevated temperature. This will facilitate moves from hard core to soft core overlap. When the system is free from hard core overlap, we continue with a “regular” equilibration at the target temperature, which in turn is followed by a production run

Useful tips

- Make sure to check for energy conservation, as described in the lecture notes. This is often a sensitive check for errors in the code.
- Do *not* recalculate the full interaction energy (an $o(N^2)$ calculation) in order to evaluate the requested ΔU , when making an attempted move. It suffices to calculate how the chosen particle interacts with the surrounding ones, at the “old” and “new” positions (an $o(N)$ calculation).
- The particles of this model contains a hard core and attempted moves that result in hard core overlap will never be accepted. It is therefore computationally wise to immediately exit any ΔU calculation loop when such an overlap is found, and reject the attempted move

A special note for Python users

While you are free to use whatever programming language you prefer, we know that many of you are mainly familiar with Python. However, this language is not designed for computationally expensive calculations, so there are some special considerations that you should be aware of (if you use Python). For-loops in Python are particularly time consuming if we have a for-loop within another for-loop (nested for-loop). We can find a 10X or more speed up with the use of array libraries and slicing arrays, in order to avoid nested for-loops.

Use numpy or pandas, as shown below, in order for simulations to be run in reasonable time. The following code is only an example solution, you are also free to utilize other methods of calculation so long as they produce the desired results.

We have the periodic boundary condition (PBC) code in python as:

```
dx = dx - np.round(dx/Length)*Length
dy = dy - np.round(dy/Length)*Length
```

Giving all dx, dy, and dz values as $-L/2 \leq dx, dy \leq L/2$. The negative values are nothing to worry about, as they are all squared for the potential calculations: $dr^2 = dx^2 + dy^2$.

The first two U functions listed are for the full system energy calculations. Whereas the last function, “*U_for_1_particle()*”, provides the potential energy contribution for one particle with all others. This will save considerable time in MC simulations when checking energy differences for the movement of one particle in the system.

U calculation, brute force double for-loop: DO NOT use the following type of double loops (nested loops). Very slow, but it illustrates the associated performance issues.

```
# x and are numpy arrays of length number of particles.
# With x,y positions for each particle.
# "Length" is the box length.
```

"identity" is an integer identifying a chosen molecule.

```
def U_full_Double_loop(x,y,Length):
    result = 0
    for i in range(0,np.size(x)):
        for j in range(i+1,np.size(x)):
            dx = x[i]-x[j]
            dy = y[i]-y[j]
            ...
            #Add PBC code
            ...
            phi = ... #Calculate potential for this pair of particles
            result += phi
    return result
```

U calculation using numpy array slicing: Simplifies to only one for-loop. Sum over values for every particle compared, because phi will now be an array instead of a double

```
def U_full_slicing(x,y,Length):
    result = 0
    for d in range(0,np.size(x)-1):
        dx = (x[d] - x[d+1:])
        dy = (y[d] - y[d+1:])
        ...
        #Add PBC code
        ...
        phi = ... #Calculate potentials for particle
        result += np.sum(phi)
    return result
```

U calculation for energy contribution for movement of a single particle: Where the "identity" of the particle integer is also a variable input for the function. Using numpy arrays and slicing, allows for no loops to be used. Sum over values for every particle compared, because phi will now be an array instead of a double.

```
def U_for_1_particle(x,y,Length,identity):
    result = 0
    dx = (x[identity] - x[np.arange(len(x))!= identity])
    dy = (y[identity] - y[np.arange(len(y))!= identity])
    ...
    #Add PBC code
    ...
    phi =... #Calculate potentials for particle with all other
    result += np.sum(phi)
    return result
```

Additionally, these are other useful modules you will likely need:

```
import pandas as pd
```

```

import numpy as np
import random
import time
import matplotlib.pyplot as plt

#pandas:
pd.read_csv( )
pd.to_csv( )

#numpy:
np.savetxt( )
np.loadtxt( )

#random, generate random integer or random double
random.randrange( )
random.uniform( )

#time, to see how long simulation takes
time.time()

#For plotting, pandas or matplotlib are useful
plt.plot()
plt.show()

```

The project

We shall start by investigating simulated structures at various densities and a fixed temperature. We then turn our focus to a structural phase transition that is brought about by a gradual lowering of the temperature, as a fixed density. In the original paper, the authors investigated systems containing 1000 particles, but in order to reduce the requested simulation time you may choose smaller systems, with (say) $N = 200$, or even $N = 100$.

Note that you will have to write the code efficient enough to sample at least $\sim 10^7$ attempted configurations, for the production runs (preferably $\sim 10^8$). If this requires unreasonable simulation times, you need to rewrite the code so that it runs more efficiently (see tips above).

Various densities, at $kT/\epsilon = 0.1$

Here we shall adopt a fixed “reduced” (unitless) temperature, $T^* = kT/\epsilon = 0.1$, where kT is the thermal energy (k is Boltzmann’s constant). It is convenient to set σ_0 as our unit of length, and define a reduced density $\rho^* \equiv N\sigma_0^2/L^2$, where L is the length of the quadratic simulation box.

Establish typical structures at the following densities: $\rho^* = 0.10, 0.15, 0.227, 0.291$ and 0.38 . Illustrate the structures, for each system, in two different ways:

- by plotting a “typical” 2D configurational snapshot
- by plotting the radial distribution function, $g(r) = \langle n(r) \rangle / n_b$, where $\langle n(r) \rangle$ is the average density within a circular strip at a distance r from a “tagged” particle at the origin ($r = 0$) whereas $n_b = N/L^2$ is the overall average density of the system.

In order to keep things apart, it might be useful to run the simulations in separate directories, one for each density.

Simulations of a gradual temperature drop, at $\rho^* = 0.291$

Start at $T^* = 0.25$ and run a sequence of gradually dropping temperatures ($0.25, 0.24 \dots, 0.16, 0.15$), saving the final coordinates (after a production run) at each temperature. This is the most time consuming part of the project, and here you might have to settle with a small system ($N = 100?$). Comment on any observed differences. This is not a course in Statistical Mechanics, but it turns out that kT^2 times the heat capacity, C_v , can be written as an energy fluctuation:

$$kT^2 C_v = \langle U^2 \rangle - \langle U \rangle^2$$

, where U is the total interaction energy. Moreover, C_v displays a divergence (for macroscopic system) at first-order phase transitions. Monitor $\langle U^2 \rangle - \langle U \rangle^2$ at each simulated temperature, and show that these results indicate a phase transition. You do not have to be concerned with units - focus instead on relative changes, i.e. the *shape* of the curve. Remember to, at each temperature, first run an equilibrium simulation, where averages are *not* sampled. Typically speaking, an equilibrium simulation is about 10-20% as long as a production run.